PYTHON PROGRAMMING

NAME: M.Sharmila

REG NO:192572139

GITHUB LINK:

https://github.com/sharmilamohan12/python/tree/main/CO%20TOOLS

ASSIGNMENT 2 – CO2

CO2 Coverage: Analyse control flow constructs, string operations and functions to perform string manipulations – if, if–else, if–elif–else, while, for, break, continue and pass, functions with parameters, return values, recursion and scope, and string operations.

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50

1.Problem Statement

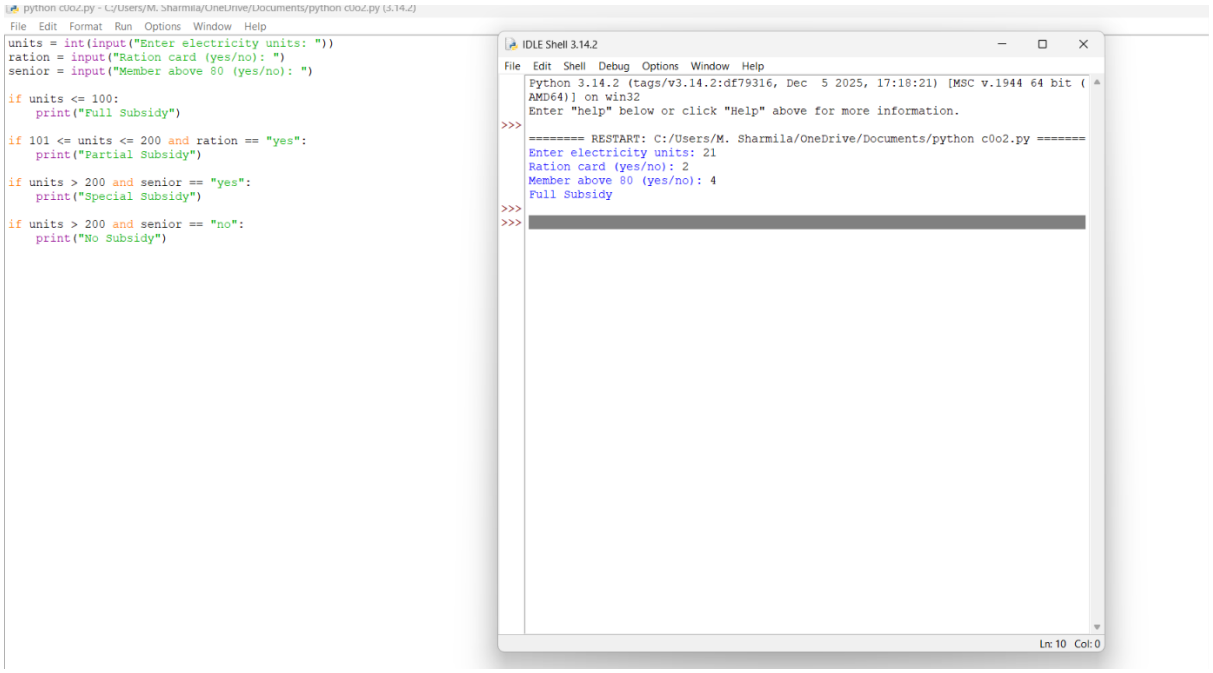
To determine household electricity subsidy eligibility based on electricity consumption, ration card availability, and senior citizens above 80 years.

2. Predictions Before Execution

Household	Units	Ration Card	Member Above 80	Expected Decision
H1	80	Yes	No	Full Subsidy
H2	100	No	No	Full Subsidy
H3	101	Yes	No	Partial Subsidy
H4	150	No	No	No Subsidy
H5	200	Yes	No	Partial Subsidy
H6	200	No	No	No Subsidy
H7	201	No	Yes	Special Subsidy

Household Units	Ration Card Member Above 80	Expected Decision
H8	250 Yes No	No Subsidy

Code &output:



Q2 – Loops that Count Attempts

Bloom's Level: K3 – Apply

1. Problem Statement

To implement OTP verification in a bank kiosk with three attempts. Blank entries should not count, and the account should be locked after three incorrect attempts.

A. Code & output

```
while attempts < 3:
    otp = input("Enter OTP: ")

    if otp == "":
        print("Blank entry! Try again.")
        continue

    attempts += 1

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break
    else:
        print("Incorrect OTP!")

else:
    print("Account Locked for 30 minutes!")
```

```
==
Enter OTP: 
2
Incorrect OTP!
Enter OTP: 2
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
```

b.using for loop

```
correct_otp = "1234"

for i in range(3):
    otp = input("Enter OTP: ")

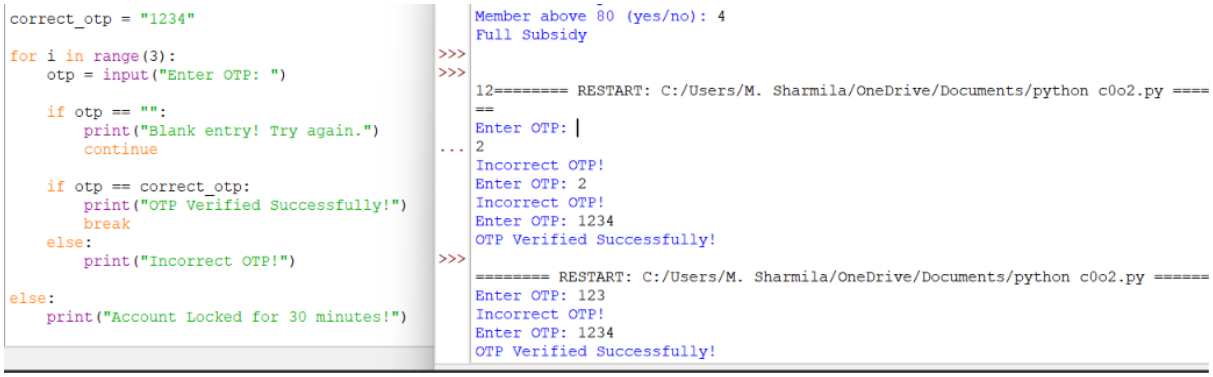
    if otp == "":
        print("Blank entry! Try again.")
        continue

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break
    else:
        print("Incorrect OTP!")

else:
    print("Account Locked for 30 minutes!")
```

```
Member above 80 (yes/no): 4
Full Subsidy
>>>
>>>
12======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py ====
==
Enter OTP: 
... 2
Incorrect OTP!
Enter OTP: 2
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
>>>
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py ======
Enter OTP: 123
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
```

c.Use of Break, Continue, Pass and Else



```
correct_otp = "1234"

for i in range(3):
    otp = input("Enter OTP: ")

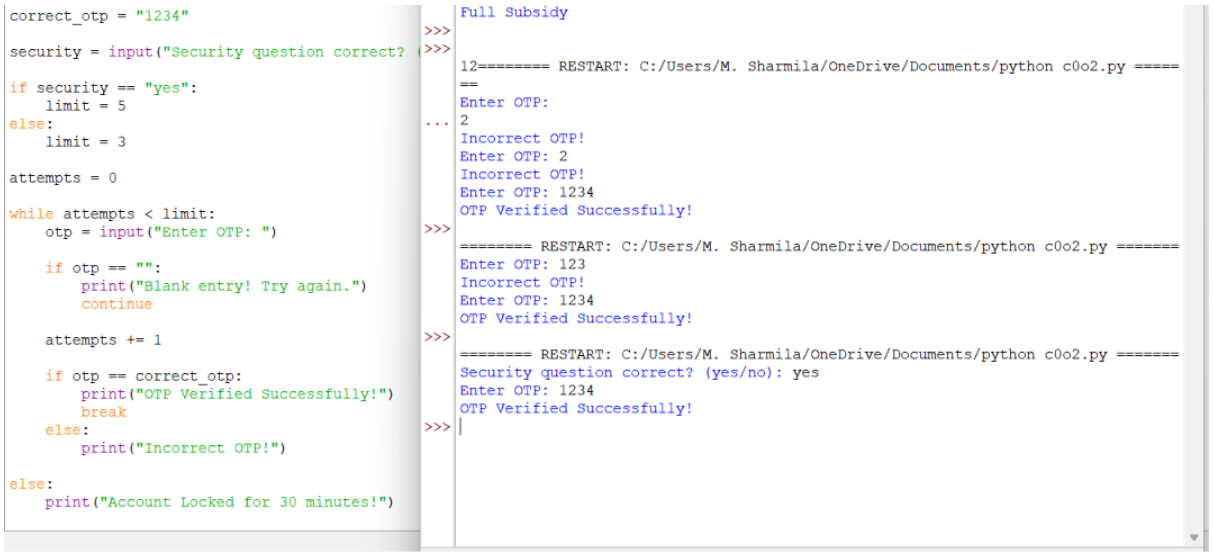
    if otp == "":
        print("Blank entry! Try again.")
        continue

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break
    else:
        print("Incorrect OTP!")

else:
    print("Account Locked for 30 minutes!")
```

```
Member above 80 (yes/no): 4
Full Subsidy
>>>
>>>
12======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py ====
==
Enter OTP:
... 2
Incorrect OTP!
Enter OTP: 2
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
>>>
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py ======
Enter OTP: 123
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
```

d. changed condition



```
correct_otp = "1234"

security = input("Security question correct? (

if security == "yes":
    limit = 5
else:
    limit = 3

attempts = 0

while attempts < limit:
    otp = input("Enter OTP: ")

    if otp == "":
        print("Blank entry! Try again.")
        continue

    attempts += 1

    if otp == correct_otp:
        print("OTP Verified Successfully!")
        break
    else:
        print("Incorrect OTP!")

else:
    print("Account Locked for 30 minutes!")
```

```
Full Subsidy
>>>
>>>
12======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py =====
==
Enter OTP:
... 2
Incorrect OTP!
Enter OTP: 2
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
>>>
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py =======
Enter OTP: 123
Incorrect OTP!
Enter OTP: 1234
OTP Verified Successfully!
>>>
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py =======
Security question correct? (yes/no): yes
Enter OTP: 1234
OTP Verified Successfully!
>>>
```

3. Engineer a Vehicle Registration Number Validator

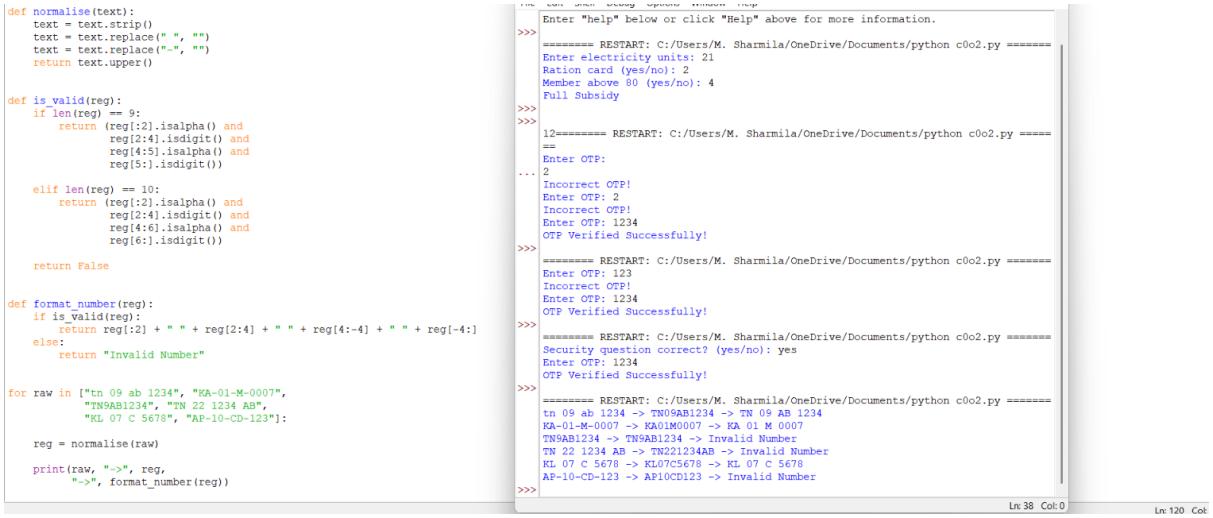
1. Problem Statement

To develop a Python program that normalises, validates and formats vehicle registration numbers using string operations.

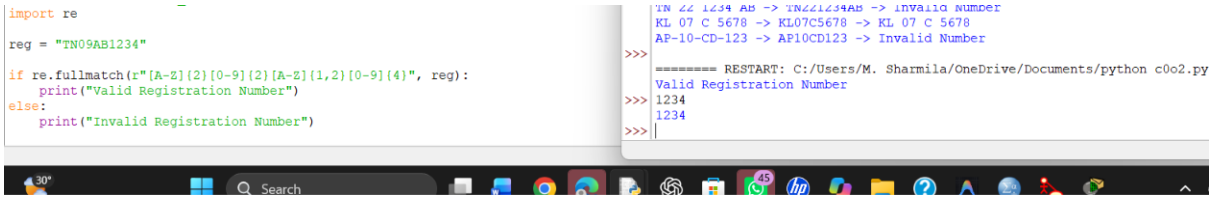
String Immutability

- Strings in Python are immutable.
- It means their original characters cannot be modified.
- text.upper() returns a new uppercase string without changing the original string.
- We must store the result using text = text.upper().

IMPLEMENTATION:



B. Alternative Concept – Regular Expression

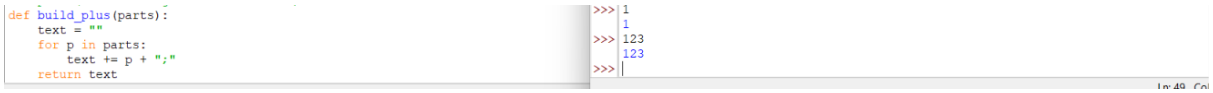


4. Building Long Text: + or join()?

1. Problem Statement

To generate a monthly SMS bill statement using different string-building methods and compare their execution time.

A.USING REPEATED +=



B. Using join()

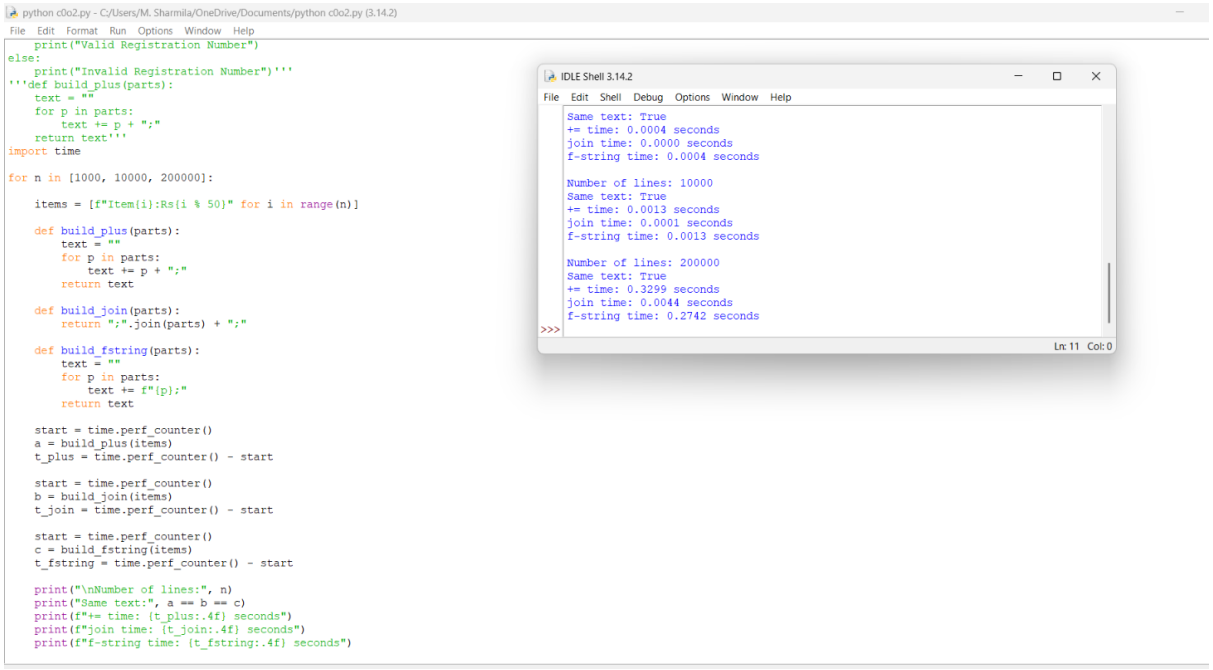
```
def build_join(parts):
    return ";".join(parts) + ";"
```

C. Using F-String

```
def build_fstring(parts):
```

```
    text = ""
    for i, p in enumerate(parts):
        text += f"{p};"
    return text
```

D. Complete Benchmark Program



```
    print("Valid Registration Number")
else:
    print("Invalid Registration Number")'''
'''def build_plus(parts):
    text = ""
    for p in parts:
        text += p + ";"
    return text'''
import time

for n in [1000, 10000, 200000]:

    items = [f"Item{i}:Rs{i % 50}" for i in range(n)]

    def build_plus(parts):
        text = ""
        for p in parts:
            text += p + ";"
        return text

    def build_join(parts):
        return ";".join(parts) + ";"

    def build_fstring(parts):
        text = ""
        for p in parts:
            text += f"{p};"
        return text

    start = time.perf_counter()
    a = build_plus(items)
    t_plus = time.perf_counter() - start

    start = time.perf_counter()
    b = build_join(items)
    t_join = time.perf_counter() - start

    start = time.perf_counter()
    c = build_fstring(items)
    t_fstring = time.perf_counter() - start

    print("\nNumber of lines:", n)
    print("Same text:", a == b == c)
    print(f"+= time: {t_plus:.4f} seconds")
    print(f"join time: {t_join:.4f} seconds")
    print(f"f-string time: {t_fstring:.4f} seconds")
```

```
IDLE Shell 3.14.2
File  Edit  Shell  Debug  Options  Window  Help
Same text: True
+= time: 0.0004 seconds
join time: 0.0000 seconds
f-string time: 0.0004 seconds

Number of lines: 10000
Same text: True
+= time: 0.0013 seconds
join time: 0.0001 seconds
f-string time: 0.0013 seconds

Number of lines: 200000
Same text: True
+= time: 0.3299 seconds
join time: 0.0044 seconds
f-string time: 0.2742 seconds
>>>
Ln: 11  Col: 0
```

Q5 – Functions, Parameters and Scope

Bloom's Level: K4 – Analyse

1. Problem Statement

To develop a canteen order system using Python functions, parameters and scope while avoiding shared default-list errors.

Explanation of the Bug

In Python, default values are created only once when a function is defined.

- order=[] creates one shared list.
- Every function call uses the same list.
- Previous customers' items appear in the next customer's order.

Solution: Use None as the default value and create a new list inside the function

A.BUGGY FUNCTION

IMPLEMENTATION:

```
def add_item_buggy(item, order=[]):
    order.append(item)
    return order

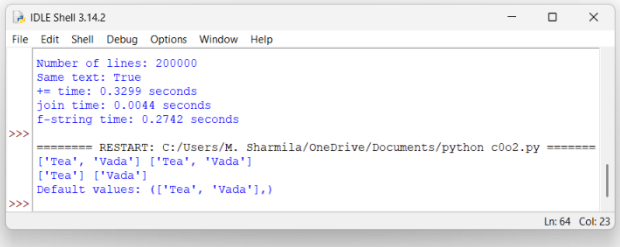

def add_item(item, order=None):
    if order is None:
        order = []

    order.append(item)
    return order


print(add_item_buggy("Tea"), add_item_buggy("Vada"))

print(add_item("Tea"), add_item("Vada"))

print("Default values:", add_item_buggy.__defaults__)
```



B.CORRECTED FUNCTION

```
def add_item(item, order=None):
    if order is None:
        order = []

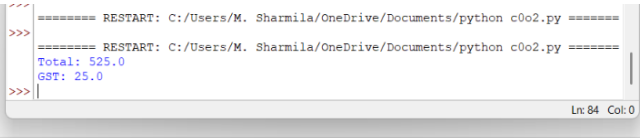
    order.append(item)
    return order
```

C. RETURNING MULTIPLE VALUES

```
def calculate(bill):
    gst = bill * 0.05
    total = bill + gst
    return total, gst

total, gst = calculate(500)

print("Total:", total)
print("GST:", gst)
```



D. TYPES OF VARIABLE

```
x = 100  # Global variable

def outer():
    y = 50  # Enclosing variable

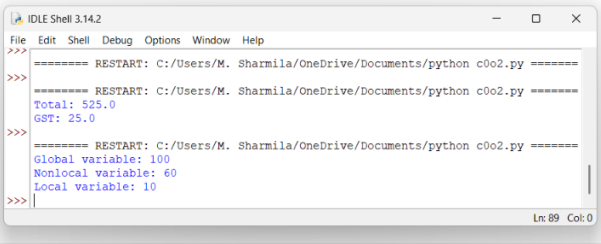
    def inner():
        nonlocal y
        y = 60  # Modifies enclosing variable

        z = 10  # Local variable

        print("Global variable:", x)
        print("Nonlocal variable:", y)
        print("Local variable:", z)

    inner()

outer()
```



Conclusion

- Default mutable lists can cause shared-data bugs.
- Using None creates independent lists.
- Positional, keyword and default parameters improve flexibility.
- Local, global and nonlocal variables control scope.

- Independent order lists make the canteen system reliable.

Q6 – Recursion and Its Limits

Bloom's Level: K5 – Evaluate

1. Problem Statement

To calculate the number of different ways to climb temple steps by taking either 1 or 2 steps at a time using recursion, memoisation and loops.

Recursive Solution:

```
uter()'''
ef ways_naive(n):
    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)
```

```
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py =======
Global variable: 100
Nonlocal variable: 60
Local variable: 10
```

Memoisation Using Dictionary Cache

```
from functools import lru_cache

calls = 0

def ways_naive(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n-1) + ways_memo(n-2)


calls = 0
print("naive:", ways_naive(25), "calls =", calls)

calls = 0
print("memo :", ways_memo(25), "calls =", calls)
```

```
IDLE Shell 3.14.2
File  Edit  Shell  Debug  Options  Window  Help
======== RESTART: C:/Users/M. Sharmila/OneDrive/Documents/python c0o2.py =======
naive: 121393 calls = 242785
memo : 121393 calls = 26
>>>
```

Alternative Concept – Using Loop

```
def ways_loop(n):
    if n <= 1:
        return 1

    a = 1
    b = 1

    for i in range(2, n+1):
        c = a + b
        a = b
        b = c

    return b


print("Ways for 1000 steps:", ways_loop(1000))
```

7. Diagnose, Redesign and Validate

PSEUDO CODE:

START

Read n

Set a = 1, b = 1

Repeat from 2 to n:

c = a + b

a = b

b = c

Print b

STOP

Implement and Validate the Redesign

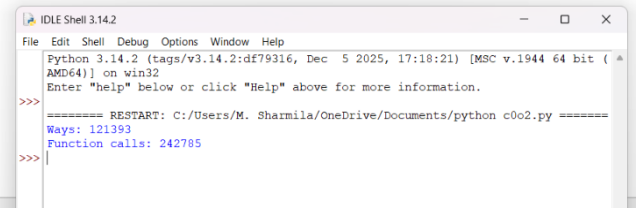
```
calls = 0

def ways_naive(n):
    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)


print("Ways:", ways_naive(25))
print("Function calls:", calls)
```



Redesigned Program – Iteration

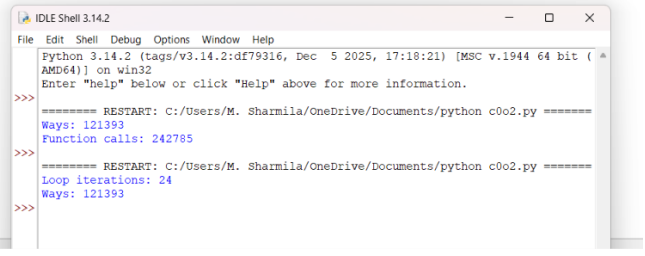
```
def ways_loop(n):
    if n <= 1:
        return 1

    a = 1
    b = 1
    count = 0

    for i in range(2, n+1):
        c = a + b
        a = b
        b = c
        count += 1

    print("Loop iterations:", count)
    return b


print("Ways:", ways_loop(25))
```



Conclusion

The original recursive program was inefficient because it repeated calculations. The redesigned iterative program uses only two variables and a loop to calculate the result. It reduces time complexity from $O(\varphi^n)$ to $O(n)$ and space complexity from $O(n)$ to $O(1)$.

Final Decision: The iterative approach is more suitable for calculating stair-climbing possibilities for up to 1,000 steps because it is faster, memory-efficient and avoids recursion errors.